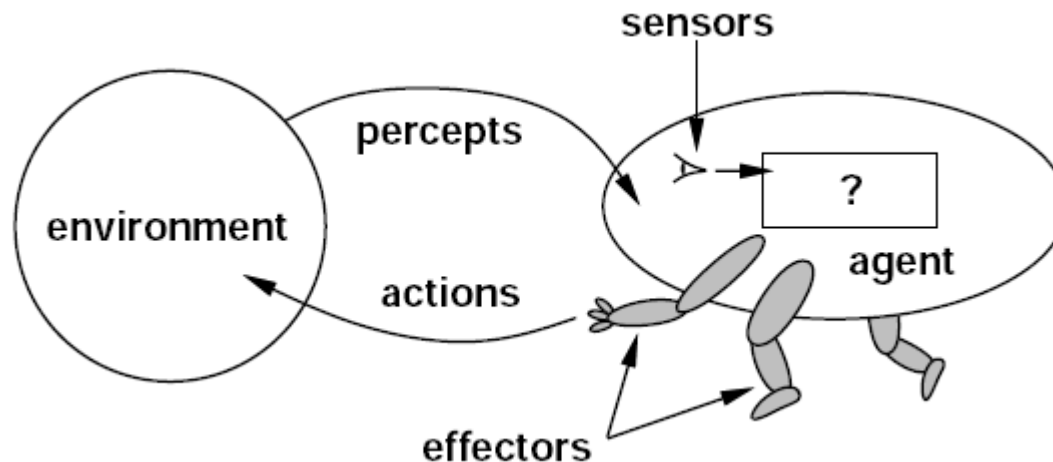


First agent steps

**Introducing agent programming
in MARBEL**

Recap: Intelligent Agents

An **agent** is anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that **environment** through **effectors**.

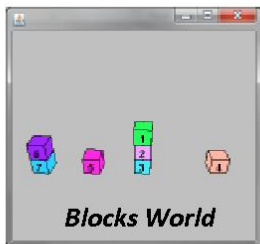


Internally, an agent is: **agent = architecture + program.**

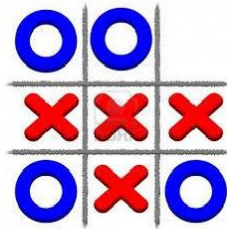
Core business of AI:
designing agent architectures & programs.

Environments

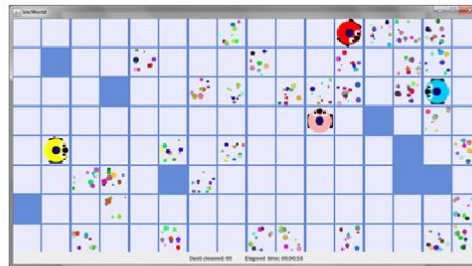
Agents are **situated** in an environment.



Blocks World



Tic Tac Toe



Vacuum World



Starcraft



Emohawk Virtual Environment



Nao Robot



Unreal Tournament

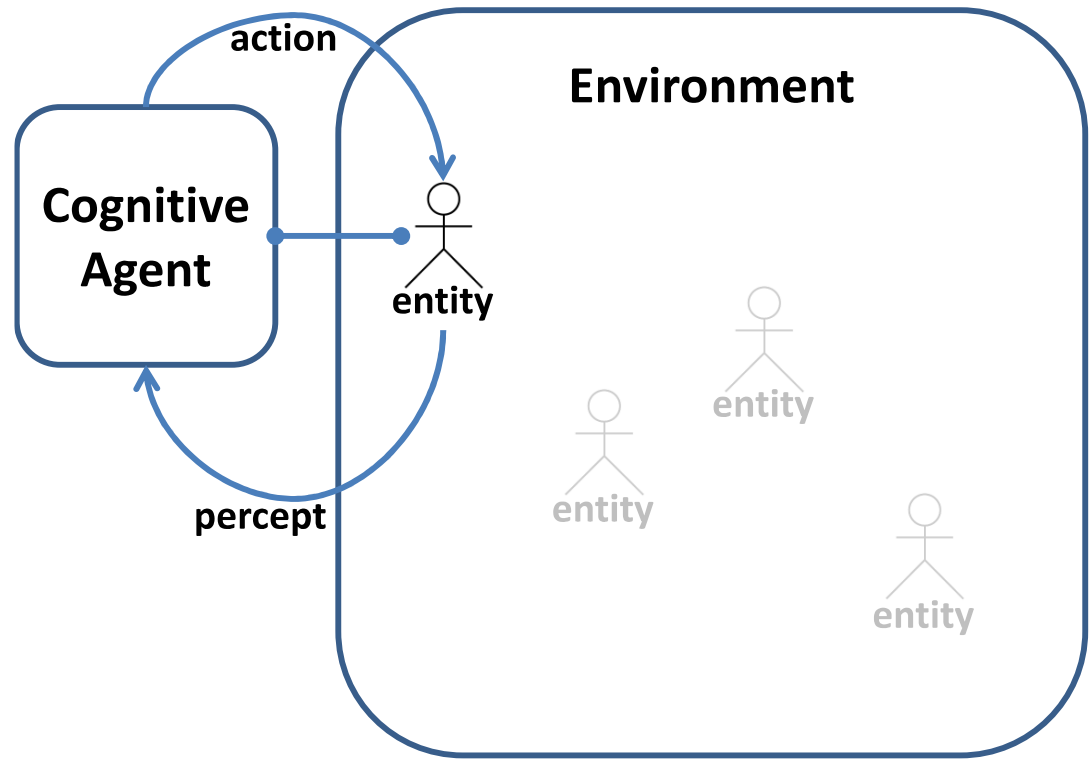


Traffic Simulator

Environment-properties:
It is very important to know the **characteristics of an environment**, before designing an agent.

Agents & Environments

For agents we take the view that an environment offers **controllable entities** which are connected to agents.



Perspective:

- **Agent** is the **mind**
- **Controllable entity** is the **body**.

Environment defines which **actions** are available and when.

Learning objectives

Introduce and gain a basic understanding of the core elements of MARBEL agents:

- MAS file: **agent definition** and **launch policy**.
- The **agent execution cycle**.
- MARBEL basics:
 - **use clause** for Prolog files;
 - **built-in and environment actions**;
 - **action rules**;
 - **order of rule execution**.
- MAS file: **use of an environment**.
- Use of **basic debug perspective features**.

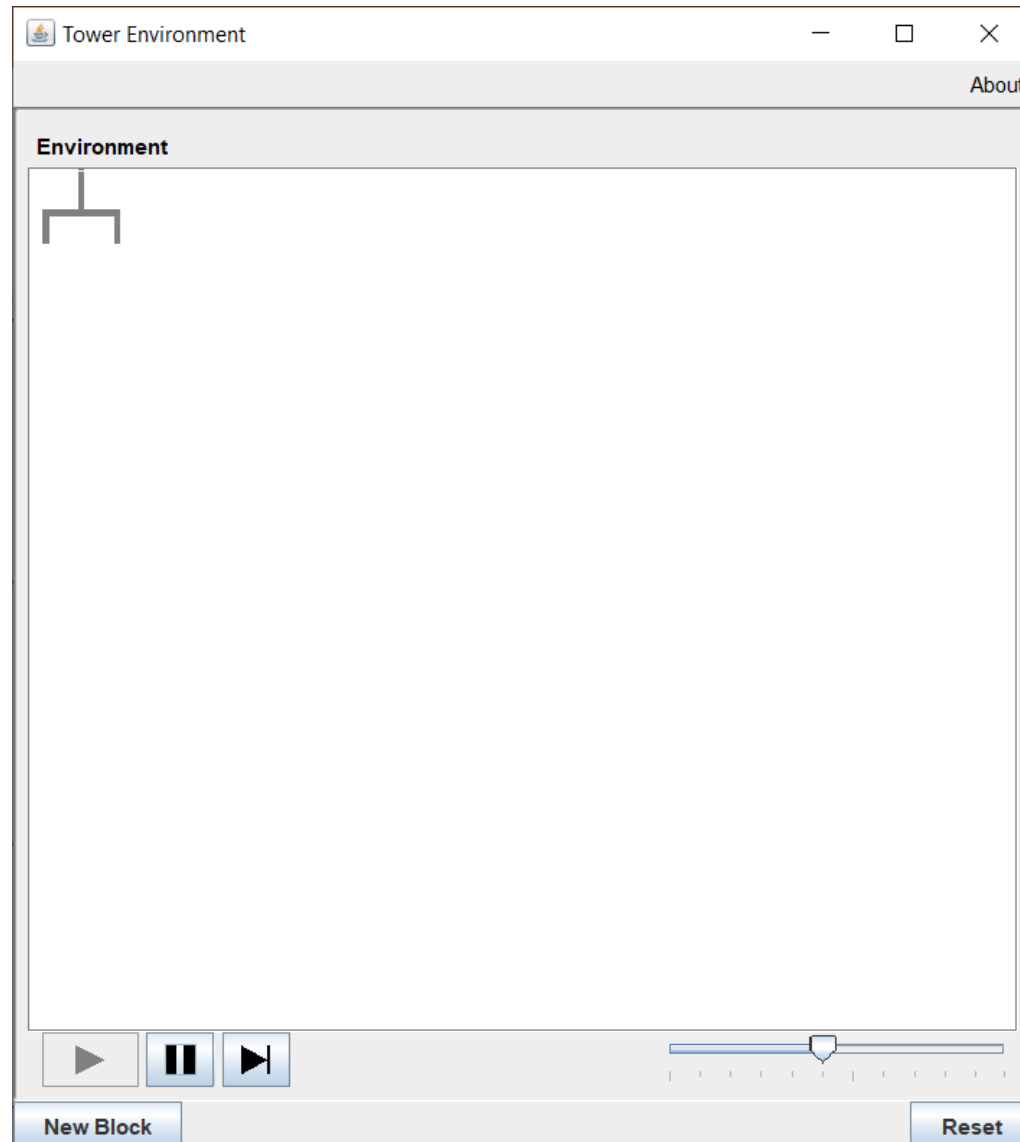
Agenda

- Select, launch, and run an example project: Tower World.
- Walk through Hello World example: modules & rules.
- Walk through Script agent example: environment & channels.
- Show how to launch the BW4T server.
- Show BW4T initial agent & mas school solution.

Select & launch example project

- In Eclipse, open the MARBEL perspective:
 - Select Window → Perspective → Open Perspective → Other → MARBEL.
- In the script explorer, right click → New → MARBEL Example Project, and select Tower.
- Unfold the project and right click on the mas2g file, select Debug as → MARBEL.
- Open the MARBEL Debug Perspective.

The Tower World



Run example project: Tower World

- Add 10 New Blocks (a-j) in Tower Environment.
- In the Debug perspective, click on the MARBEL Debugging Engine.
- Press the Resume button (or F8).
- Use a mouse to move blocks around yourself.
- Pause the Debugging engine and inspect the console and introspector.

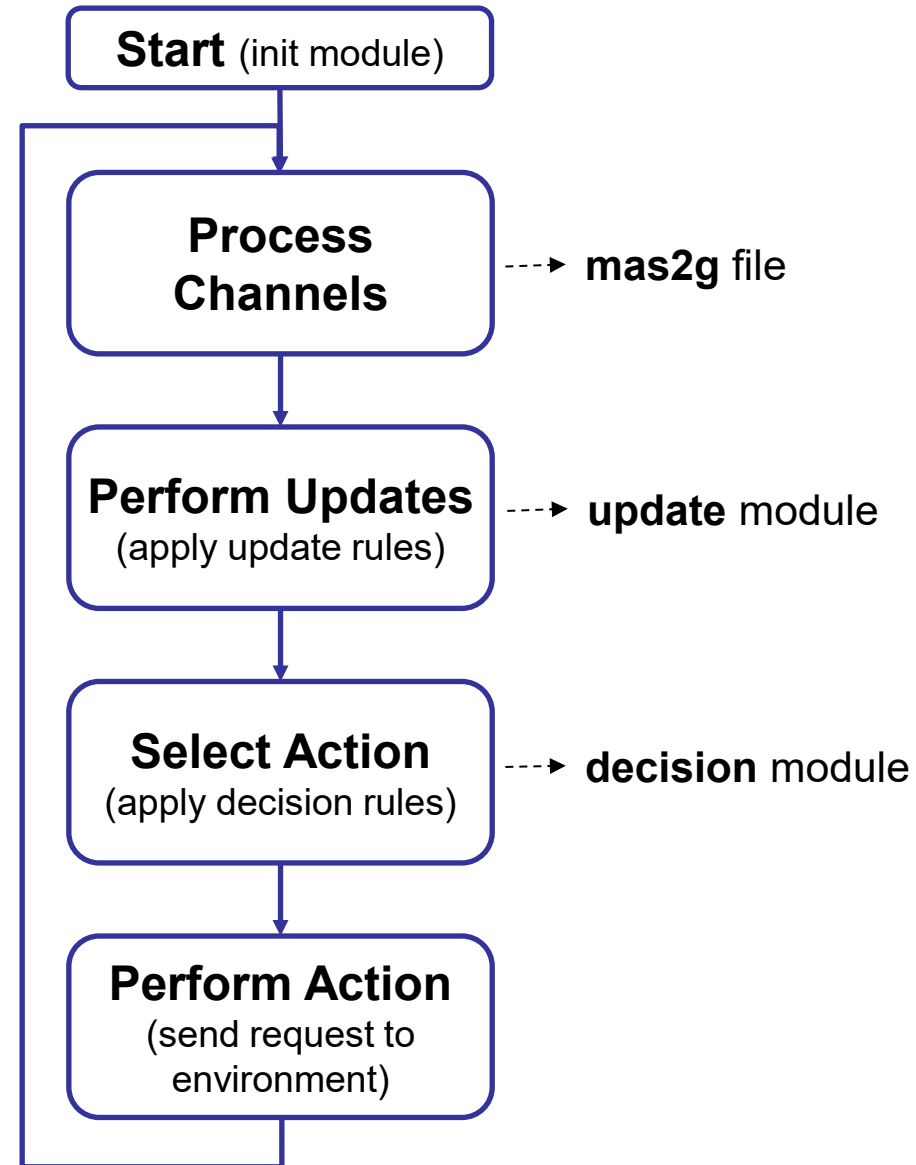
The Tower World Environment

1. **Fully observable**: agent can see “everything” relevant for decision making.
2. **Deterministic**: action effects are predictable.
3. **Dynamic**: environment can change even if agent does nothing because user can interact via GUI.
4. **Discrete**: finite list of blocks; nb: continuous gripper movement not relevant for decision making.
5. **Multi agent**: there is only a single gripper entity to connect agent to but also a user who can do things.

➤ We need sensing (percepts) because of 3.

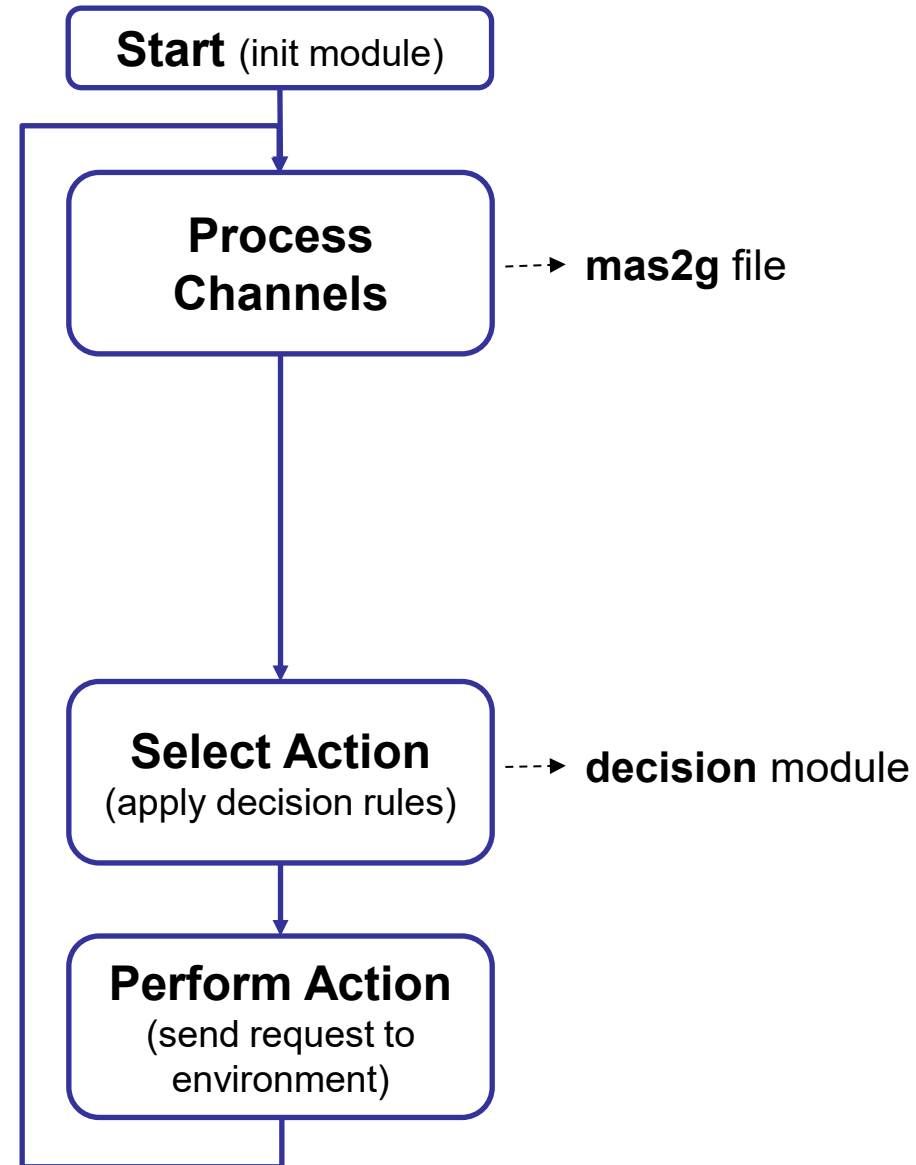
Agent Execution Cycle

- Start: **init** module:
 - Initialize database state
- Process: **channels**
 - Process percepts
 - Process messages
- Process: **update** module
 - Update database state
- Select: **decision** module
 - Action selection (strategy)
- Perform: **action**:
 - Request env to execute action



Agent Cycle: Tower World Example

- Start: **init** module:
 - Initialize database state
- Process: **channels**
 - Process percepts
 - ~~Process messages~~
- ~~Process: **update** module~~
 - ~~Update database state~~
- Select: **decision** module
 - Action selection (strategy)
- Perform: **action**:
 - Request env to execute action



A HELLO WORLD AGENT

A MAS FILE

Must have:

- An agent definition:

```
define helloWorldAgent {  
    use helloWorld for decisions.  
}
```

Module name

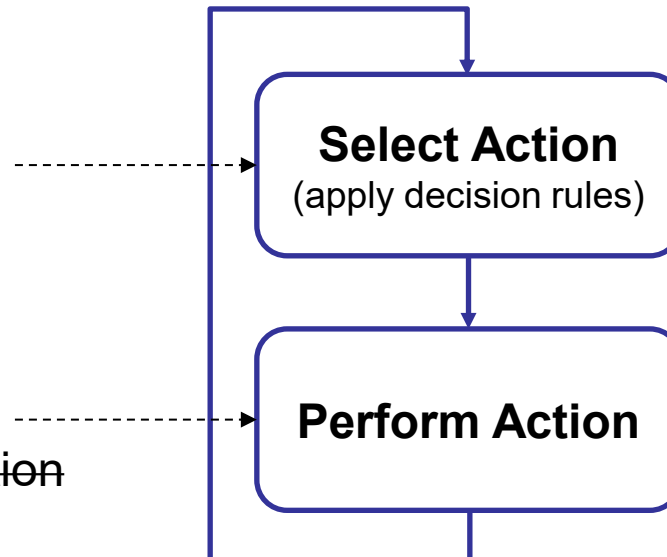
Agent's name

- A launch policy:

```
launchpolicy {  
    launch helloWorldAgent.  
}
```

Agent Cycle: Hello World

- ~~Start: **init** module:~~
 - ~~— Initialize database state~~
- ~~Process: **channels**~~
 - ~~— Process percepts~~
 - ~~— Process messages~~
- ~~Process: **update** module~~
 - ~~— Update database state~~
- Select: **decision** module
 - Action selection (strategy)
- Perform: ***built-in* action:**
 - ~~— Request env to execute action~~



Run HelloWorld.mas2g

- Agent not connected to any environment.
- Inspect agent console. What happened?
- Change logging preferences:
 - Select Window → Preferences → MARBEL → Logging; check “Built-in actions”.
- Run again. Inspect agent console.

Inside a Module: helloWorld.mod2g

Must have:

- A **use clause** (identifies that Prolog is used)
- A named **module** section with at least one...
- ... **action rule**:
 if *true* then **print**("Hello, world!").

Action rules

Example: **if** *true* **then** **print**("Hello, world!").

Form: **if** <Prolog query> **then** <action>.

- Any Prolog query can be used.
- Actions can be either built-in or provided by an environment; **print** is a built-in action.

Comparing Action & Prolog rules

Action rule in MARBEL agent:

Form: **if** <Prolog query> **then** <action>.

Example: **if** $X=4$ **then** **print**(X).

Prolog rule:

Form: <head(VarList)> :- <Prolog query>.

Example: $p(X) :- X=4$.

Action rules with variables

Can use **variables** just like Prolog rules:

if $X=4$ then print(X).

Try instead of $X=4$:

- $X=\text{helloWorld}$
- $X=[1, 2, 3]$
- $X=1+2$

Action rules with variables

Try:

if $X=4$ then print(Y).

if $X=[Y, 2]$ then print(X).

Check out Eclipse & Problems console.

All variables used in action part of an action rule MUST be used AND instantiated (grounded) by query of the rule.

- No database needed to evaluate Prolog queries on previous slides!
 - true
 - $X=4$
 - ...

all are equal to output returned in Prolog console.

Run MultipleLines.mas2g

- Agent not connected to any environment.
- Inspect agent console. What happened?

Agent performed multiple cycles.

But what happened? → Breakpoints

- Add a breakpoint to the rule in the init module `initCounter.mod2g`.
- Step through the code.
- Inspect the:
 - agent console,
 - variable evaluation pane, and
 - introspector (agent's state).

Modules can have parameters too

```
module helloWorldMultipleLines(Lines) {  
    if printedText(Count), Count < Lines then print("Hello, world!").  
}
```

Module parameters are instantiated everywhere where they appear in the module:

```
module helloWorldMultipleLines(10) {  
    if printedText(Count), Count < 10 then print("Hello, world!").  
}
```

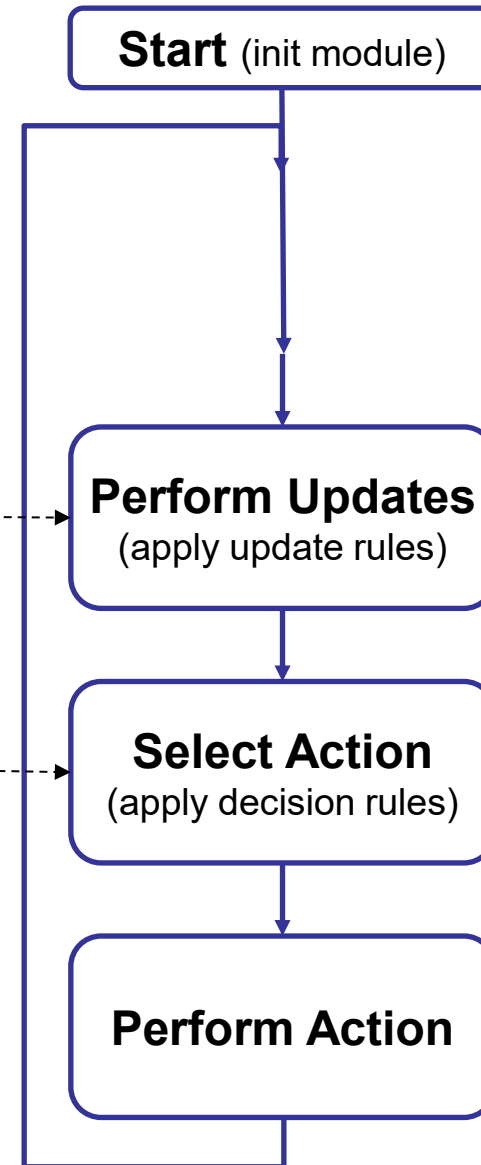
Module parameters for setting a goal

```
module helloWorldMultipleLines(Lines) {  
    if printedText(Count), Count < Lines then print("Hello, world!")  
}
```

The module parameter is used to **set a goal**:
How many times to print a “Hello, world!” line.

Agent Cycle: MultipleLines Example

- Start: **init** module:
 - Initialize database state
- ~~Process: **channels**~~
 - ~~Process percepts~~
 - ~~Process messages~~
- Process: **update** module
 - Update database state
- Select: **decision** module
 - Action selection (strategy)
- Perform: **action**:
 - ~~Request env to execute action~~



Queries are evaluated on the database

Database:

printedText(0)

Query:

printedText(Count), NewCount is Count+1

Evaluation:

Count/0, NewCount/1.

Here Count/0 means that Count = 0 is true.

Checkout the Interactive Console for evaluating queries.

Built-in actions for updating database

Action part of rule:

delete(printedText(0)) + **insert**(printedText(1))

Database:

printedText(0).

Built-in actions for updating database

After performing **delete**(printedText(0))

Remaining action part is:

insert(printedText(1))

Database:

is empty.

Built-in actions for updating database

After performing

delete(printedText(0)) + **insert**(printedText(1))

There is no remaining action part.

Database:

printedText(1).

+ operator can be used to combine actions in a rule.

Declaring predicates used

```
module helloWorldMultipleLines(Lines) {  
    if printedText(Count), Count < Lines then print("Hello, world!");  
}
```

- Module uses printedText/1 predicate.

Predicates used MUST be declared in a Prolog file that is used in the module.

Declaring predicates used

In file `counter.pl`:

`:- dynamic` `printedText/1`.

General form:

`:- dynamic`

`predicate1/<Arity1>, predicate2/<Arity2>, ... .`

Example:

`:- dynamic` `printedText/1, on/2`.

USING AN ENVIRONMENT

Run ScriptAgent.mas2g

- Agent connected to an environment.
- Agent initially paused in debug mode.

Use clause for environment

In mas2g file, at the top:

use "HelloWorldEnvironment-1.3.0.jar".

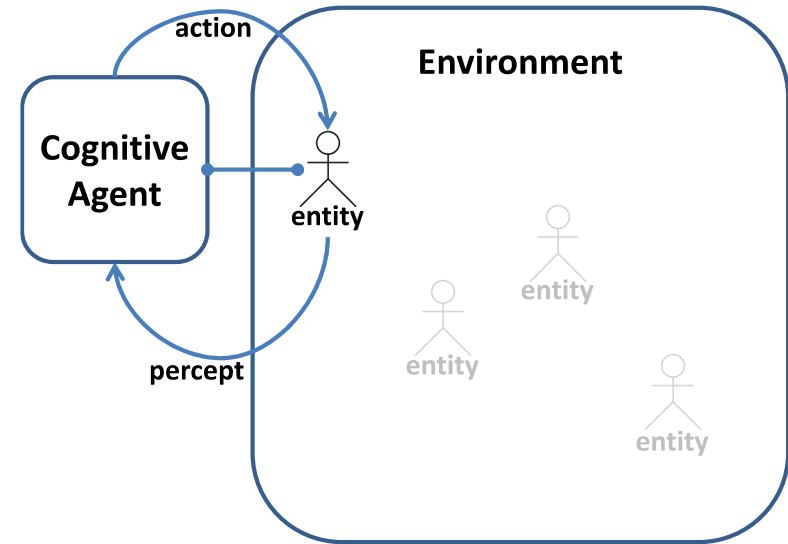
General form use clause:

use "<connectorname>.jar".

***Connector can also launch environment itself;
depends on environment.***

Subscribing to percepts

In order to receive percepts, an agent needs to subscribe to the corresponding **channels**.

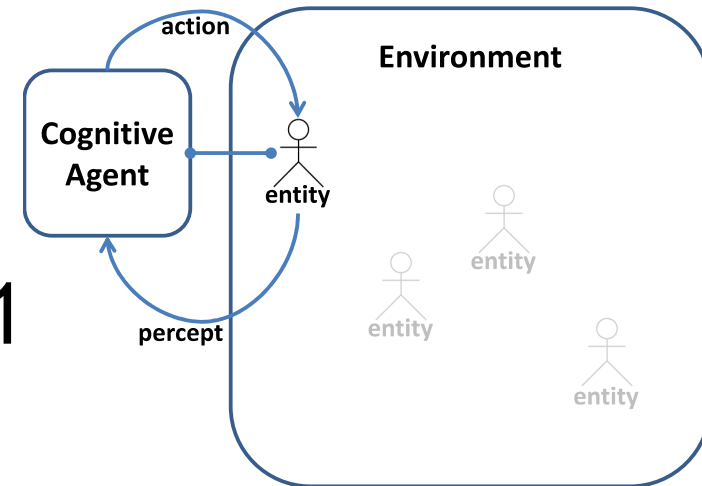


In agent df in mas2g file, example:
replace lastPrintedText/1.
replace printedText/1.

Subscribing to percepts

In mas2g file, example:

replace lastPrintedText/1
replace printedText/1.



Which percept (predicates) are available depends on environment; read the manual ;-).

Connecting an agent to an entity

```
launchpolicy {  
    when * launch helloWorldAgent.  
}
```

Meaning:

- when entity becomes available in environment,
- then: launch an agent, and
- connect it to this entity.

Agent will receive percepts via this entity.

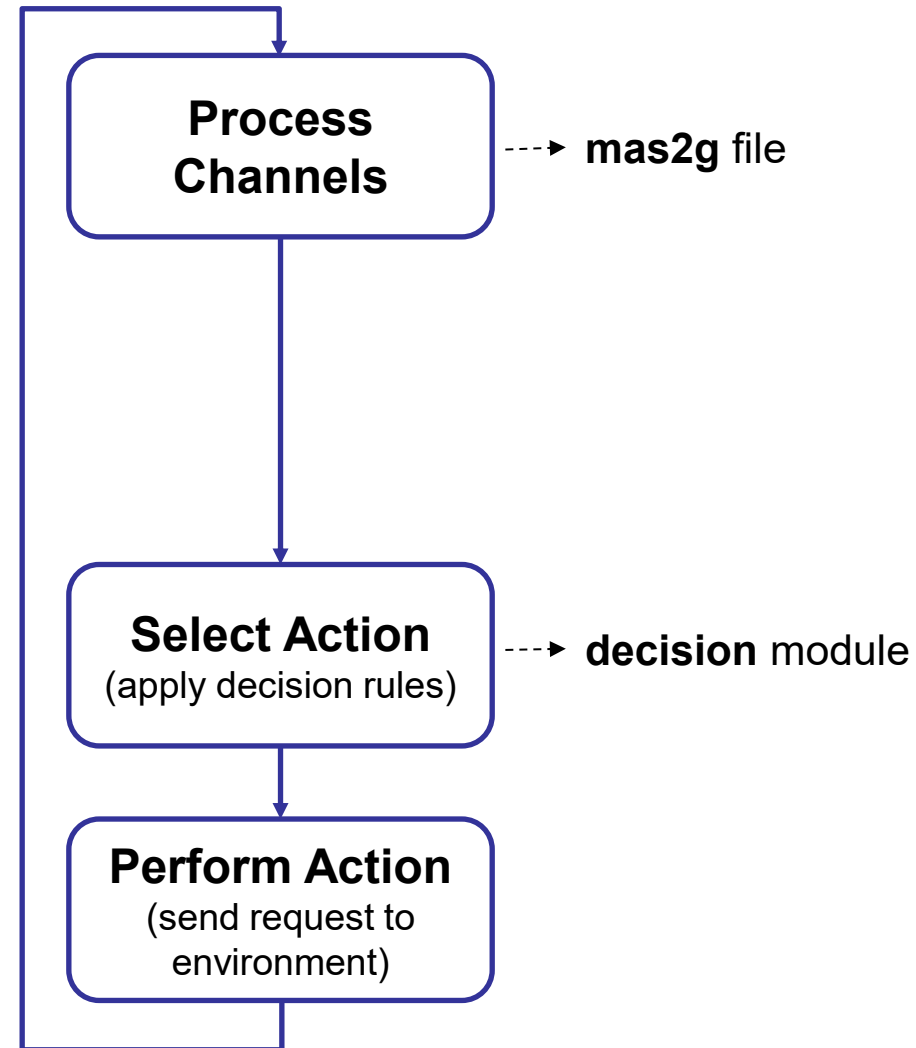
No need for init & update module

```
define helloWorldAgent {  
    use printScript(1 1) for decisions.  
  
    replace lastPrintedText/1.  
    replace printedText/1.  
}
```

*Percepts from environment are **automatically processed** and update agent's database!*

Agent Cycle: ScriptAgent Example

- ~~Start: **init** module:~~
 - ~~— Initialize database state~~
- Process: **channels**
 - Process percepts
 - ~~— Process messages~~
- ~~Process: **update** module~~
 - ~~— Update database state~~
- Select: **decision** module
 - Action selection (strategy)
- Perform: **action**:
 - Request env to execute action



printScript module:

Multiple use clauses

use counter. % *for declaring a predicate*

use script. % *script lines, initializes database.*

exit=noaction.

module printScript(Goal) {

if printedText(X), $X < \text{Goal}$,

 NextLine is $X+1$, script(NextLine, Text)

then printText(Text).

}

SUMMARY

Agent Programming

An agent programming language is
a programming language with
events, beliefs, actions, decisions,
as first class citizens.

MARBEL Agent Core Capabilities

- **event processing:**
update its state using the events
(percepts, messages) it receives;
- **representing information:** 
represent & reason about its environment;
- **decision-making:**
select an action to perform next.

Key language elements

- **events** received from environment & agents
- **database** to represent environment 
- **actions** to update an agent's database, send messages to other agents, act in environment
- **rules** to select actions & modules
- **modules** to structure rules & action selection
- **support for multi-agent systems**

Assignment

BW4T ENVIRONMENT

Launching the BW4T server

BW4T Connector is **client** only → have to launch environment separately.

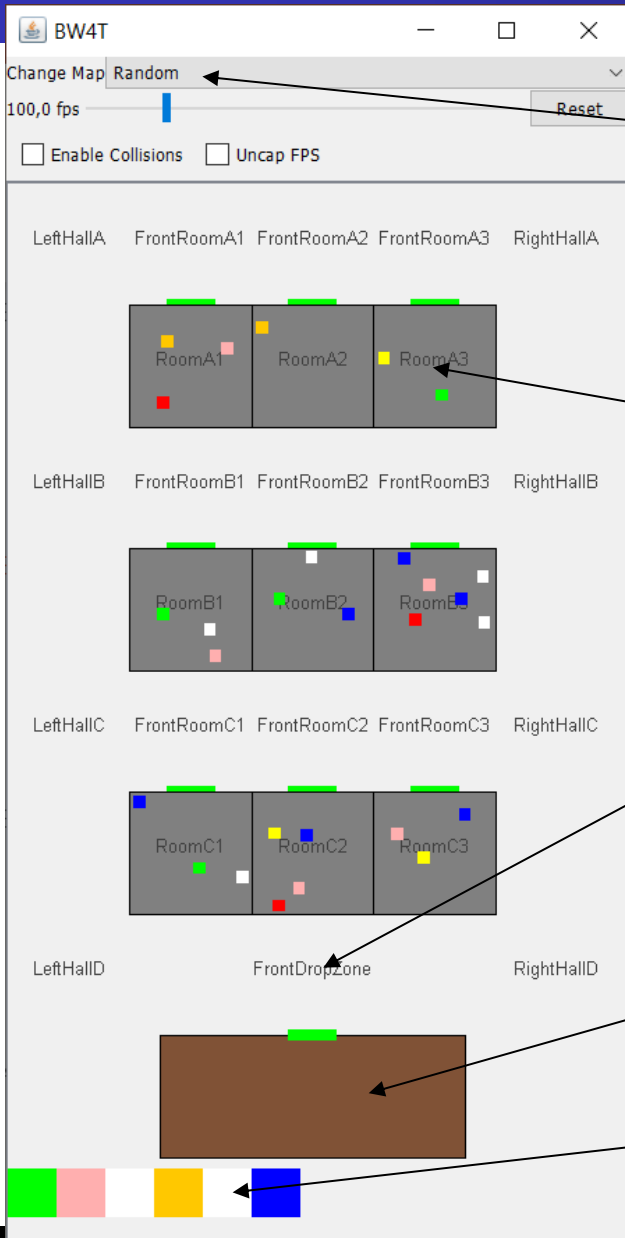
Open a terminal.

Go to the folder with the BW4T server.

Execute:

```
java -jar bw4t-server-3.10.1.jar
```

BW4T environment



**Select
different map**

**Rooms with
blocks to explore**

All robots start here

**DropZone to
deliver blocks to**

**Target goal
sequence**

Initial agent & MAS school solution

Initial agent:

Single agent that only visits all rooms in the environment.

MAS school solution:

Multiple agents that explore together and distribute the work of delivering blocks.

Organisation

- **Read** MARBEL Programming Guide [Chapters 1-2](#)
 - Also contains exercises to practice with